

# Модуль 11. Архитектура и паттерны проектирования

---

## Введение в модуль

К этому моменту мы умеем писать асинхронный код, валидировать запросы, работать с базами данных, аутентифицировать пользователей, интегрировать ML-модели и деплоить в контейнеры. Но **инженерия — это не сумма технологий, а их организация**. Можно знать все инструменты и построить систему, которая через полгода станет неподдерживаемой.

Этот модуль — про **мышление архитектора**. Мы разберём:

- почему «чистая архитектура» — это не модное слово, а способ выжить при росте кодовой базы;
- как паттерны вроде Circuit Breaker и Idempotency Key защищают систему от каскадных отказов;
- когда REST — правильный выбор, а когда он убивает производительность;
- почему CAP-теорема ограничивает *любую* распределённую систему, независимо от технологий;
- как Event-Driven архитектура решает проблемы, которые REST создаёт, и какие новые проблемы привносит.

## 11.1. Чистая архитектура и слоистая структура

### 11.1.1. Проблема: спагетти-код

Представьте типичный «быстрый» FastAPI-проект:

```
In [1]: @app.post("/predict")
        async def predict(input_data: InputModel):
            # Валидация
            if not input_data.features:
                raise HTTPException(400, "Empty features")

            # Прямой SQL-запрос
            conn = await asyncpg.connect(DATABASE_URL)
            user = await conn.fetchrow("SELECT * FROM users WHERE id = $1", input_data.use
```

```

r_id)

# Загрузка модели
model = torch.load("model.pth")

# Inference
features = torch.tensor(input_data.features)
with torch.no_grad():
    result = model(features)

# Сохранение в БД
await conn.execute(
    "INSERT INTO predictions (user_id, result) VALUES ($1, $2)",
    input_data.user_id, result.item()
)

# Отправка email
await send_email(user["email"], f"Your prediction: {result.item()}")

return {"prediction": result.item()}

```

Один endpoint делает **всё**: валидацию, доступ к БД, ML-инференс, бизнес-логику, отправку уведомлений. Через месяц:

- Нужно поменять БД — ищите SQL по всему проекту.
- Нужно протестировать логику — невозможно без поднятия PostgreSQL и загрузки модели.
- Нужно добавить кэш — меняете код в 15 местах.
- Нужно переиспользовать логику в другом endpoint — копируете 50 строк.

**Чистая архитектура** (Clean Architecture, Robert C. Martin) решает это через **разделение на слои** с жёстким правилом зависимостей.

### 11.1.2. Слои чистой архитектуры

In [2]:

|                                       |
|---------------------------------------|
| Presentation Layer (API, Controllers) |
| FastAPI endpoints, Pydantic schemas,  |
| HTTP-специфичная логика               |
| Application Layer (Use Cases)         |
| Оркестрация: «что делать», не «как»   |
| Сервисы, команды, запросы             |

|                                                                      |  |
|----------------------------------------------------------------------|--|
| Domain Layer (Entities, Business Rules)                              |  |
| Чистая бизнес-логика, независимая<br>от фреймворков и инфраструктуры |  |
| Infrastructure Layer (DB, External APIs)                             |  |
| SQLAlchemy, Redis, HTTP-клиенты,<br>email-сервисы, ML-модели         |  |

**Dependency Rule (Правило зависимостей):** зависимости направлены **внутрь**. Внешние слои зависят от внутренних, но не наоборот. Domain Layer не знает ни о FastAPI, ни о PostgreSQL, ни о Redis.

### 11.1.3. Dependency Rule: почему это работает

Формально, если обозначить слои как модули  $M_1, M_2, M_3, M_4$  (снизу вверх), то:

$$\forall i < j : M_j \text{ может импортировать } M_i, \text{ но } M_i \nrightarrow M_j$$

Это создаёт **ациклический граф зависимостей** (DAG — Directed Acyclic Graph). Ацикличность гарантирует:

- **Тестируемость:** Domain Layer тестируется без FastAPI, БД, сети — чистые unit-тесты.
- **Заменяемость:** можно поменять PostgreSQL на MongoDB, изменив только Infrastructure Layer.
- **Понятность:** читая Domain Layer, вы видите бизнес-логику, а не SQL.

### 11.1.4. Интерфейсы в Python: Protocol и абстрактные классы

Чтобы Domain Layer не зависел от Infrastructure, нужен **интерфейс** — контракт, который реализует Infrastructure, а Domain использует.

**Вариант 1:** `abc.ABC` (номинальная типизация)

```
In [3]: from abc import ABC, abstractmethod
        from typing import Any

        class PredictionRepository(ABC):
            @abstractmethod
            async def save(self, prediction: Prediction) -> None:
                ...

            @abstractmethod
```

```

    async def get_by_user(self, user_id: str) -> list[Prediction]:
        ...

class PostgresPredictionRepository(PredictionRepository):
    def __init__(self, session: AsyncSession):
        self._session = session

    async def save(self, prediction: Prediction) -> None:
        self._session.add(PredictionORM.from_domain(prediction))
        await self._session.commit()

    async def get_by_user(self, user_id: str) -> list[Prediction]:
        result = await self._session.execute(
            select(PredictionORM).where(PredictionORM.user_id == user_id)
        )
        return [r.to_domain() for r in result.scalars().all()]

```

## Вариант 2: `typing.Protocol` (структурная типизация)

```

In [4]: from typing import Protocol

class PredictionRepository(Protocol):
    async def save(self, prediction: Prediction) -> None:
        ...

    async def get_by_user(self, user_id: str) -> list[Prediction]:
        ...

class PostgresPredictionRepository:
    # Не наследует PredictionRepository!
    # Но тупо считает его совместимым, потому что методы совпадают
    def __init__(self, session: AsyncSession):
        self._session = session

    async def save(self, prediction: Prediction) -> None:
        ...

    async def get_by_user(self, user_id: str) -> list[Prediction]:
        ...

```

## Когда что использовать:

- `ABC` — когда нужна явная иерархия, runtime-проверка ( `isinstance` ).
- `Protocol` — когда нужна гибкость, loose coupling, тестовые дублёры без наследования.

## 11.1.5. Практика: структура проекта

```
In [5]: ml_api/
| domain/                                # <- Самый внутренний слой
| | __init__.py
| | entities.py                        # Prediction, User, ModelConfig
| | repositories.py                   # Protocol-интерфейсы
| | services.py                       # Чистая бизнес-логика
|
| application/                         # <- Use Cases
| | __init__.py
| | predict.py                        # PredictUseCase
| | notify.py                         # NotificationUseCase
|
| infrastructure/                     # <- Внешний мир
| | __init__.py
| | db/
| | | models.py                      # SQLAlchemy ORM
| | | repositories.py               # Реализации Protocol
| | ml/
| | | model_loader.py              # Загрузка ONNX/Torch
| | email/
| | | smtp_client.py
|
| presentation/                       # <- FastAPI
| | __init__.py
| | main.py                         # app = FastAPI()
| | routers/
| | | predict.py
| | dependencies.py                 # DI-зависимости
```

**Ключевое правило:** `domain/` не импортирует `application/`, `infrastructure/` или `presentation/`. `infrastructure/` импортирует `domain/` (чтобы реализовать интерфейсы). `presentation/` импортирует `application/` и `domain/`.

## 11.1.6. Пример: Use Case «Сделать предсказание»

```
In [6]: # domain/entities.py
from dataclasses import dataclass
from datetime import datetime

@dataclass(frozen=True)
```

```

class Prediction:
    id: str
    user_id: str
    model_version: str
    input_features: list[float]
    result: float
    created_at: datetime

# domain/repositories.py
from typing import Protocol

class PredictionRepository(Protocol):
    async def save(self, prediction: Prediction) -> None: ...
    async def get_by_user(self, user_id: str) -> list[Prediction]: ...

class ModelService(Protocol):
    async def predict(self, features: list[float], version: str) -> float: ...

# application/predict.py
from dataclasses import dataclass
from domain.entities import Prediction
from domain.repositories import PredictionRepository, ModelService
import uuid
from datetime import datetime

@dataclass
class PredictUseCase:
    prediction_repo: PredictionRepository
    model_service: ModelService

    async def execute(self, user_id: str, features: list[float], version: str) -> Prediction:
        # Чистая бизнес-логика: нет SQL, нет HTTP, нет FastAPI
        result = await self.model_service.predict(features, version)

        prediction = Prediction(
            id=str(uuid.uuid4()),
            user_id=user_id,
            model_version=version,
            input_features=features,
            result=result,
            created_at=datetime.utcnow(),
        )

        await self.prediction_repo.save(prediction)
        return prediction

```

```
# presentation/routers/predict.py
from fastapi import APIRouter, Depends
from application.predict import PredictUseCase
from presentation.dependencies import get_predict_use_case

router = APIRouter()

@router.post("/predict")
async def predict(
    input_data: PredictRequest,
    use_case: PredictUseCase = Depends(get_predict_use_case),
):
    prediction = await use_case.execute(
        user_id=input_data.user_id,
        features=input_data.features,
        version=input_data.model_version,
    )
    return {"prediction_id": prediction.id, "result": prediction.result}
```

### Преимущества:

- `PredictUseCase` можно протестировать с `MockPredictionRepository` и `MockModelService` — никакой БД, никакого PyTorch.
- Можно заменить PostgreSQL на DynamoDB — меняется только `infrastructure/db/repositories.py`.
- Можно добавить кэширование — оборачиваем `PredictionRepository` в `CachedPredictionRepository`, не трогая Use Case.

### 11.1.7. Математическая подоплека: топологическая сортировка зависимостей

Dependency Rule гарантирует, что граф зависимостей между модулями **ацикличен**.

Формально, если представить модули как вершины графа  $G = (V, E)$ , где  $E$  — зависимости (ребро  $u \rightarrow v$  означает « $u$  импортирует  $v$ »), то чистая архитектура требует:

$G$  — DAG (Directed Acyclic Graph)

Для DAG существует **топологическая сортировка** — упорядочение вершин  $v_1, v_2, \dots, v_n$  такое, что для любого ребра  $v_i \rightarrow v_j$  выполняется  $i < j$ .

В нашем случае: `domain` -> `application` -> `infrastructure` -> `presentation` — это топологическая сортировка. Она гарантирует, что:

- Можно тестировать слои по одному, снизу вверх.
- Можно заменять вершины (модули) без нарушения порядка.
- Циклические зависимости (например, `domain` импортирует `infrastructure` ) невозможны по построению.

Инструменты вроде `import-linter` или `mypy` с `--strict` помогают автоматически проверять это правило в CI.

## 11.2. Паттерны для асинхронных систем

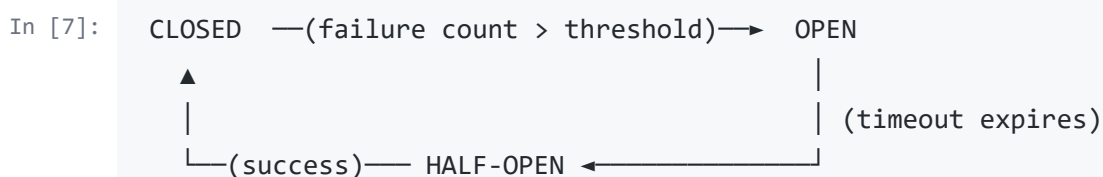
Распределённые системы ломаются. Сети разрываются, сервисы падают, базы данных перегружаются. Паттерны ниже — это **защитные механизмы**, которые делают систему устойчивой к отказам.

### 11.2.1. Circuit Breaker: предотвращение каскадных отказов

**Проблема:** сервис А вызывает сервис В. Если В падает или тормозит, запросы к А накапливаются в очереди, потоки/корутины блокируются, А тоже падает. Затем С, зависящий от А, падает. **Каскадный отказ.**

**Решение:** Circuit Breaker — автоматический выключатель. Если В не отвечает  $N$  раз подряд — «выключаем» вызовы к В на время  $T$ , сразу возвращая ошибку. Это даёт В время восстановиться и предотвращает исчерпание ресурсов А.

**Состояния:**



| Состояние        | Поведение                                                     |
|------------------|---------------------------------------------------------------|
| <b>CLOSED</b>    | Запросы проходят нормально. Счётчик ошибок = 0.               |
| <b>OPEN</b>      | Запросы немедленно отклоняются (fail fast). Таймер идёт.      |
| <b>HALF-OPEN</b> | Один тестовый запрос. Если успех -> CLOSED. Если нет -> OPEN. |

**Реализация:**



```

In [8]: import asyncio
from enum import Enum, auto
from dataclasses import dataclass, field
from typing import Callable, Any
import time

class State(Enum):
    CLOSED = auto()
    OPEN = auto()
    HALF_OPEN = auto()

@dataclass
class CircuitBreaker:
    failure_threshold: int = 5
    recovery_timeout: float = 30.0 # секунд
    expected_exception: type = Exception

    _state: State = field(default=State.CLOSED, repr=False)
    _failure_count: int = field(default=0, repr=False)
    _last_failure_time: float = field(default=0.0, repr=False)
    _lock: asyncio.Lock = field(default_factory=asyncio.Lock, repr=False)

    async def call(self, coro: Callable[..., Any], *args, **kwargs) -> Any:
        async with self._lock:
            if self._state == State.OPEN:
                if time.monotonic() - self._last_failure_time > self.recovery_time
out:
                self._state = State.HALF_OPEN
            else:
                raise CircuitBreakerOpen("Service temporarily unavailable")

        try:
            result = await coro(*args, **kwargs)
            await self._on_success()
            return result
        except self.expected_exception:
            await self._on_failure()
            raise

    async def _on_success(self):
        async with self._lock:
            self._failure_count = 0
            self._state = State.CLOSED

    async def _on_failure(self):
        async with self._lock:

```

```

        self._failure_count += 1
        self._last_failure_time = time.monotonic()
        if self._failure_count >= self.failure_threshold:
            self._state = State.OPEN

class CircuitBreakerOpen(Exception):
    pass

# Использование
breaker = CircuitBreaker(failure_threshold=3, recovery_timeout=10.0)

async def call_ml_service(features: list[float]) -> float:
    # HTTP-запрос к ML-серверу
    async with httpx.AsyncClient() as client:
        resp = await client.post("http://ml-service:8000/predict", json={"features": features})
        resp.raise_for_status()
        return resp.json()["result"]

@app.post("/predict")
async def predict(input_data: InputModel):
    try:
        result = await breaker.call(call_ml_service, input_data.features)
        return {"prediction": result}
    except CircuitBreakerOpen:
        # Fallback: используем упрощённую модель на CPU
        result = await fallback_model.predict(input_data.features)
        return {"prediction": result, "fallback": True}

```

**Библиотеки:** `pybreaker` , `aiobreaker` .

### 11.2.2. Retry с экспоненциальным backoff и jitter

**Проблема:** временный сбой (network blip, GC pause) не должен считаться фатальным. Но если все клиенты retry'ят одновременно с фиксированным интервалом — создаётся **thundering herd** (лавинная нагрузка).

**Решение:** экспоненциальный backoff + случайный jitter.

```

In [9]: import asyncio
import random
from typing import Callable, TypeVar

T = TypeVar('T')

```

```

async def retry_with_backoff(
    coro: Callable[..., T],
    max_retries: int = 3,
    base_delay: float = 1.0,
    max_delay: float = 60.0,
    exceptions: tuple = (Exception,),
) -> T:
    """
    Экспоненциальный backoff с полным jitter.

    Формула задержки перед i-й попыткой:
    delay = min(base_delay * 2^i, max_delay)
    jittered_delay = random.uniform(0, delay)
    """
    for attempt in range(max_retries + 1):
        try:
            return await coro()
        except exceptions as e:
            if attempt == max_retries:
                raise

            # Экспоненциальный backoff: 2^attempt * base_delay
            delay = min(base_delay * (2 ** attempt), max_delay)

            # Полный jitter: случайное значение от 0 до delay
            # Распределяет retry'и во времени, предотвращая синхронизацию
            jittered = random.uniform(0, delay)

            print(f"Attempt {attempt + 1} failed: {e}. Retrying in {jittered:.2f}s...")

            await asyncio.sleep(jittered)

```

**Математическая подоплека:** jitter — это **декорреляция** retry'ев. Без jitter  $N$  клиентов, упавших одновременно, будут retry'ить синхронно, создавая пики нагрузки. С jitter retry'и равномерно распределяются по интервалу  $[0, \text{delay}]$ , снижая пиковую нагрузку в  $N$  раз.

### 11.2.3. Idempotency Key: безопасные повторные запросы

**Проблема:** клиент отправляет `POST /charge` (списание средств). Запрос доходит до сервера, сервер списывает деньги, но ответ теряется по сети. Клиент retry'ит. Деньги списываются дважды.

**Решение:** Idempotency Key — уникальный идентификатор запроса. Сервер запоминает, что запрос с таким ключом уже обработан, и при повторе возвращает тот же результат.

```

In [10]:
Клиент                                Сервер
|                                     |
|-- POST /charge ----->|
|   Idempotency-Key: abc123   |
|                                     | [проверяет: abc123 есть в БД?]
|                                     | [нет -> обрабатывает, сохраняет результат]
|<-- 200 OK {charged: true} ----|
|                                     |
|-- POST /charge ----->| [ответ потерян, клиент retry'ит]
|   Idempotency-Key: abc123   |
|                                     | [да -> abc123 уже обработан]
|<-- 200 OK {charged: true} ----| [возвращает сохранённый результат]

```

## Реализация:

```

In [11]:
from fastapi import Header, HTTPException
from redis.asyncio import Redis
import json

async def idempotent_charge(
    amount: float,
    user_id: str,
    idempotency_key: str = Header(...),
    redis: Redis = Depends(get_redis),
):
    # Проверяем, есть ли результат в Redis
    cached = await redis.get(f"idempotency:{idempotency_key}")
    if cached:
        return json.loads(cached) # Возвращаем сохранённый результат

    # Обрабатываем (критическая секция – атомарность!)
    # Используем Redis SET NX (set if not exists) для атомарности
    lock_acquired = await redis.set(
        f"idempotency_lock:{idempotency_key}",
        "1",
        nx=True, # only if not exists
        ex=60,   # TTL 60 секунд
    )

    if not lock_acquired:
        # Другой процесс уже обрабатывает – ждём
        for _ in range(30):
            await asyncio.sleep(0.1)
        cached = await redis.get(f"idempotency:{idempotency_key}")

```

```

        if cached:
            return json.loads(cached)
        raise HTTPException(409, "Request is being processed")

    try:
        # Выполняем бизнес-логику
        result = await process_charge(amount, user_id)

        # Сохраняем результат с TTL (например, 24 часа)
        await redis.setex(
            f"idempotency:{idempotency_key}",
            86400,
            json.dumps(result),
        )
        return result
    except Exception:
        # При ошибке удаляем lock, чтобы можно было retry'ить
        await redis.delete(f"idempotency_lock:{idempotency_key}")
        raise

```

#### Ключевые свойства:

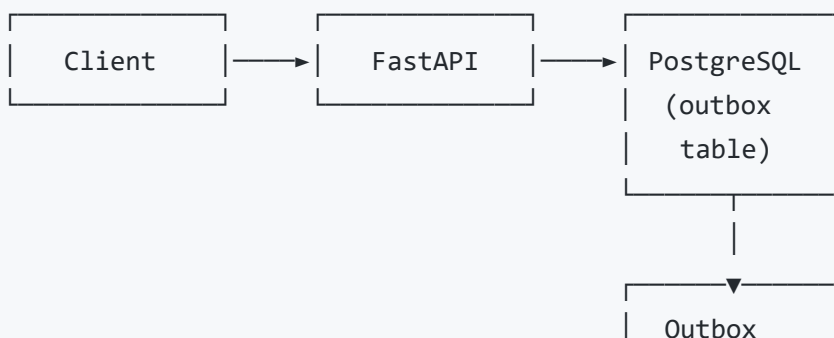
- **Идемпотентность:**  $f(f(x)) = f(x)$ . Повторный вызов с тем же ключом даёт тот же результат.
- **Атомарность:** проверка и сохранение должны быть атомарными (Redis `SET NX` или транзакция БД).
- **TTL:** результаты хранятся ограниченное время, чтобы не раздувать хранилище.

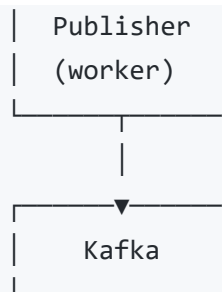
### 11.2.4. Outbox Pattern: надёжная публикация событий

**Проблема:** нужно сохранить данные в БД и опубликовать событие в Kafka. Если БД коммитится, а Kafka недоступен — событие потеряно. Если Kafka публикует, а БД откатывается — событие опубликовано, но данных нет.

**Решение:** Outbox Pattern — публикуем события **транзакционно** через БД.

In [12]:





## Схема:

```
In [13]: CREATE TABLE outbox (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    topic VARCHAR(255) NOT NULL,  
    payload JSONB NOT NULL,  
    headers JSONB,  
    created_at TIMESTAMP DEFAULT NOW(),  
    published_at TIMESTAMP,  
    retry_count INT DEFAULT 0  
);  
  
CREATE INDEX idx_outbox_unpublished ON outbox(published_at) WHERE published_at IS  
NULL;
```

## Публикация (в рамках транзакции):

```
In [14]: from sqlalchemy.ext.asyncio import AsyncSession  
from domain.events import PredictionMade  
  
async def make_prediction(  
    use_case: PredictUseCase,  
    session: AsyncSession,  
    event_bus: EventBus,  
) -> Prediction:  
    # Бизнес-логика  
    prediction = await use_case.execute(...)  
  
    # Создаём событие  
    event = PredictionMade(  
        prediction_id=prediction.id,  
        user_id=prediction.user_id,  
        result=prediction.result,  
    )
```

```

# Сохраняем в outbox ТОЙ ЖЕ транзакцией
outbox_record = OutboxORM(
    topic="predictions.made",
    payload=event.model_dump(),
    headers={"correlation-id": get_correlation_id()},
)
session.add(outbox_record)

# Коммит — и данные, и событие атомарны
await session.commit()

return prediction

```

### Outbox Publisher (фоновый worker):

```

In [15]: async def outbox_publisher(
    session_factory: async_sessionmaker,
    kafka_producer: AIOKafkaProducer,
    poll_interval: float = 1.0,
):
    while True:
        async with session_factory() as session:
            result = await session.execute(
                select(OutboxORM)
                .where(OutboxORM.published_at.is_(None))
                .order_by(OutboxORM.created_at)
                .limit(100)
                .with_for_update(skip_locked=True) # <- не блокируем другие work
er'ы
            )
            records = result.scalars().all()

            for record in records:
                try:
                    await kafka_producer.send(
                        topic=record.topic,
                        value=json.dumps(record.payload).encode(),
                        headers=[(k, v.encode()) for k, v in record.headers.items
()]),
                    )
                    record.published_at = datetime.utcnow()
                except Exception:
                    record.retry_count += 1
                    if record.retry_count > 10:
                        # DLQ: Dead Letter Queue

```

```

        record.published_at = datetime.utcnow() # помечаем, но в
DLQ

        await send_to_dlq(record)

        await session.commit()

        await asyncio.sleep(poll_interval)

```

### Почему это работает:

- БД — единственный источник истины. Если коммит прошёл — событие гарантированно будет опубликовано (eventually).
- `with_for_update(skip_locked=True)` позволяет запускать несколько publisher'ов параллельно без конфликтов.
- Если Kafka упал — publisher просто ждёт и retry'ит. Событие не потеряно.

## 11.2.5. Saga: управление распределёнными транзакциями

**Проблема:** ACID-транзакции работают только в рамках одной БД. В микросервисной архитектуре операция может затрагивать несколько сервисов. Как гарантировать консистентность?

**Saga** — последовательность локальных транзакций, где каждая транзакция обновляет данные в одном сервисе и публикует событие для следующей. Если шаг падает — выполняются **компенсирующие транзакции** (откаты).

### Пример: заказ в ML-платформе

```

In [16]: Шаг 1: Списать средства (Billing Service)
          |→ Успех -> публикуем "PaymentCharged"
          |→ Провал -> откат не нужен (деньги не списаны)

          Шаг 2: Запустить обучение (Training Service) [подписан на "PaymentCharged"]
          |→ Успех -> публикуем "TrainingStarted"
          |→ Провал -> компенсация: "RefundPayment"

          Шаг 3: Отправить уведомление (Notification Service) [подписан на "TrainingStarte
          d"]
          |→ Успех -> done
          |→ Провал -> не критично, retry

```

### Типы Saga:



| Тип                | Механизм                                                  | Плюсы                             | Минусы                               |
|--------------------|-----------------------------------------------------------|-----------------------------------|--------------------------------------|
| <b>Хореография</b> | Каждый сервис слушает события и решает, что делать дальше | Децентрализованно, loose coupling | Сложно отследить поток, отладка      |
| <b>Оркестрация</b> | Центральный оркестратор управляет шагами                  | Явный поток, легче отлаживать     | Точка отказа (оркестратор), coupling |

## Реализация хореографии:

```
In [17]: # Billing Service
@app.post("/charge")
async def charge(request: ChargeRequest):
    result = await billing_service.charge(request.user_id, request.amount)
    await event_bus.publish("PaymentCharged", {
        "order_id": request.order_id,
        "user_id": request.user_id,
        "amount": request.amount,
    })
    return result

# Training Service (подписчик)
async def on_payment_charged(event: dict):
    try:
        job = await training_service.start_training(
            user_id=event["user_id"],
            order_id=event["order_id"],
        )
        await event_bus.publish("TrainingStarted", {
            "order_id": event["order_id"],
            "job_id": job.id,
        })
    except Exception:
        # Компенсация: запрос на возврат средств
        await event_bus.publish("PaymentRefundRequested", {
            "order_id": event["order_id"],
            "reason": "training_failed",
        })
```

**Важно:** Saga — это **eventual consistency**. Между шагами система находится в промежуточном состоянии. Клиент должен быть готов к этому (например, видеть статус «обработка» вместо

финального результата).

## 11.3. API-дизайн

### 11.3.1. REST: ресурсы, HTTP-методы, статус-коды

REST (Representational State Transfer) — архитектурный стиль, а не протокол. Его принципы:

1. **Ресурсы** — всё есть ресурс ( `/users` , `/predictions` , `/models` ).
2. **Идентификаторы** — URI однозначно идентифицирует ресурс ( `/users/123` ).
3. **HTTP-методы** — семантика операции:
  - `GET` — чтение (идемпотентно, безопасно).
  - `POST` — создание.
  - `PUT` — полная замена (идемпотентно).
  - `PATCH` — частичное обновление.
  - `DELETE` — удаление (идемпотентно).
4. **Статус-коды** — результат операции:
  - `200 OK` — успех.
  - `201 Created` — создан ресурс.
  - `204 No Content` — успех, тело пустое.
  - `400 Bad Request` — ошибка клиента (невалидные данные).
  - `401 Unauthorized` — не аутентифицирован.
  - `403 Forbidden` — нет прав.
  - `404 Not Found` — ресурс не найден.
  - `409 Conflict` — конфликт (например, duplicate key).
  - `422 Unprocessable Entity` — семантическая ошибка (Pydantic validation).
  - `429 Too Many Requests` — rate limit.
  - `500 Internal Server Error` — ошибка сервера.
  - `503 Service Unavailable` — сервис временно недоступен (retry after).

#### Пример REST API для ML-платформы:

```
In [18]: GET    /models          -> список моделей
          GET    /models/{id}     -> конкретная модель
          POST   /models          -> загрузить новую модель
          PATCH  /models/{id}     -> обновить метаданные
```

```
DELETE /models/{id} -> удалить модель

POST /predictions -> создать предсказание (async)
GET /predictions/{id} -> статус/результат предсказания
GET /predictions?user_id=123 -> фильтрация по пользователю
```

### 11.3.2. RPC-style: когда уместен

REST хорош для CRUD, но плох для **операций**, которые не отображаются на ресурсы.

Примеры:

- `POST /models/{id}/deploy` — развёртывание модели.
- `POST /predictions/{id}/cancel` — отмена предсказания.
- `POST /users/{id}/retrain` — запуск переобучения.

Это **RPC-style** (Remote Procedure Call) поверх HTTP. Альтернативы:

- **gRPC** — бинарный протокол на HTTP/2, строгая типизация через protobuf. Идеален для межсервисного взаимодействия (API Gateway <-> ML Server).
- **GraphQL** — клиент запрашивает нужные поля. Хорош для сложных фронтендов, но добавляет сложность на бэкенде.

#### Выбор для ML-сервиса:

- **REST** — публичное API, документация через OpenAPI.
- **gRPC** — внутреннее взаимодействие (FastAPI <-> Triton), высокая производительность, streaming.
- **GraphQL** — если фронтенд запрашивает разные сочетания полей (редко для ML API).

### 11.3.3. Версионирование API

**Проблема:** клиенты (мобильные приложения, сторонние интеграции) не обновляются мгновенно. API должен поддерживать старые версии.

Способы:

| Способ | Пример                                                         | Плюсы               | Минусы       |
|--------|----------------------------------------------------------------|---------------------|--------------|
| URL    | <code>/v1/predictions</code> ,<br><code>/v2/predictions</code> | Явно,<br>кэшируется | URL меняется |

| Способ      | Пример                                 | Плюсы      | Минусы                  |
|-------------|----------------------------------------|------------|-------------------------|
| Заголовок   | Accept:<br>application/vnd.api.v2+json | URL чистый | Сложнее документировать |
| Query param | /predictions?api-version=2             | Просто     | Загромождает URL        |

**Рекомендация:** URL-версионирование для публичного API. Явно, понятно, работает с любым HTTP-клиентом.

```
In [19]: from fastapi import APIRouter

v1_router = APIRouter(prefix="/v1")
v2_router = APIRouter(prefix="/v2")

@v1_router.post("/predict")
async def predict_v1(data: PredictRequestV1):
    # Старая версия
    ...

@v2_router.post("/predict")
async def predict_v2(data: PredictRequestV2):
    # Новая версия с batching
    ...
```

### 11.3.4. Пагинация, фильтрация, сортировка

**Пагинация:**

```
In [20]: from fastapi import Query

@app.get("/predictions")
async def list_predictions(
    page: int = Query(1, ge=1),
    page_size: int = Query(20, ge=1, le=100),
):
    offset = (page - 1) * page_size
    # SQL: LIMIT page_size OFFSET offset
    ...
```

**Проблема offset-based:** при больших offset'ах ( `OFFSET 1000000` ) PostgreSQL сканирует миллион строк. **Cursor-based pagination** эффективнее:

```
In [21]: @app.get("/predictions")
async def list_predictions_cursor(
    cursor: str | None = None, # base64-encoded last_id
    page_size: int = Query(20, ge=1, le=100),
):
    # WHERE id > decoded_cursor ORDER BY id LIMIT page_size
    ...
```

### Фильтрация:

```
In [22]: @app.get("/predictions")
async def list_predictions(
    user_id: str | None = None,
    model_version: str | None = None,
    status: PredictionStatus | None = None,
    created_after: datetime | None = None,
):
    # Строим WHERE динамически
    ...
```

### Сортировка:

```
In [23]: @app.get("/predictions")
async def list_predictions(
    sort_by: str = "created_at",
    sort_order: Literal["asc", "desc"] = "desc",
):
    # Валидация sort_by против белого списка
    if sort_by not in {"created_at", "result", "model_version"}:
        raise HTTPException(400, "Invalid sort field")
    ...
```

## 11.3.5. OpenAPI: генерация спецификации и клиентских SDK

FastAPI генерирует OpenAPI (Swagger) спецификацию автоматически из типов Pydantic:

```
In [24]: @app.post(
    "/predict",
    response_model=PredictionResponse,
    status_code=201,
    summary="Create a prediction",
    description="Runs inference on the specified model with given features.",
    tags=["predictions"],
)
async def predict(data: PredictRequest):
    ...
```

## Польза:

- **Документация:** `/docs` — интерактивная Swagger UI.
- **Клиентские SDK:** `openapi-generator` генерирует TypeScript, Go, Java клиенты из спецификации.
- **Валидация контракта:** тесты проверяют, что реализация соответствует спецификации.

```
In [25]: # Генерация TypeScript клиента
openapi-generator-cli generate \
  -i http://localhost:8000/openapi.json \
  -g typescript-fetch \
  -o ./client
```

# 11.4. Микросервисы и распределённые системы

## 11.4.1. Монолит vs микросервисы

| Критерий                      | Монолит                    | Микросервисы                               |
|-------------------------------|----------------------------|--------------------------------------------|
| <b>Размер команды</b>         | < 10 разработчиков         | > 15 разработчиков                         |
| <b>Скорость развёртывания</b> | Всё сразу                  | Независимо                                 |
| <b>Масштабирование</b>        | Всё приложение             | Отдельные сервисы                          |
| <b>Сложность</b>              | Низкая (одна кодовая база) | Высокая (сеть, observability, consistency) |
| <b>Отказоустойчивость</b>     | Всё падает вместе          | Изоляция отказов                           |

| Критерий             | Монолит | Микросервисы      |
|----------------------|---------|-------------------|
| Технологический стек | Единый  | Разный под задачу |

**Закон Конвея (Conway's Law):** организации проектируют системы, которые копируют структуру коммуникаций внутри организации. Если у вас 3 команды — скорее всего, будет 3 сервиса (хорошо или плохо).

**Рекомендация для ML-сервиса:** начните с **модульного монолита** (чистая архитектура внутри одного deployable unit). Выделите микросервисы, когда:

- Разные части имеют разные требования к масштабированию (API — CPU, ML — GPU).
- Команды достаточно велики для автономии.
- Нужны независимые циклы релиза.

### 11.4.2. Межсервисное взаимодействие

| Протокол       | Формат   | Когда использовать                                |
|----------------|----------|---------------------------------------------------|
| REST/HTTP      | JSON     | Публичное API, простота                           |
| gRPC           | protobuf | Внутреннее, высокая производительность, streaming |
| Message Broker | События  | Асинхронность, буферизация, Event-Driven          |

**gRPC пример:**

```
In [26]: // ml_service.proto
syntax = "proto3";

service MLService {
    rpc Predict(PredictRequest) returns (PredictResponse);
    rpc StreamPredict(stream PredictRequest) returns (stream PredictResponse);
}

message PredictRequest {
    string model_id = 1;
    repeated float features = 2;
}

message PredictResponse {
    float result = 1;
```

```
float confidence = 2;
}
```

```
In [27]: # Клиент gRPC в FastAPI
import grpc
from generated import ml_service_pb2, ml_service_pb2_grpc

async def call_ml_service(model_id: str, features: list[float]) -> float:
    async with grpc.aio.insecure_channel("ml-service:50051") as channel:
        stub = ml_service_pb2_grpc.MLServiceStub(channel)
        request = ml_service_pb2.PredictRequest(
            model_id=model_id,
            features=features,
        )
        response = await stub.Predict(request)
        return response.result
```

### Преимущества gRPC:

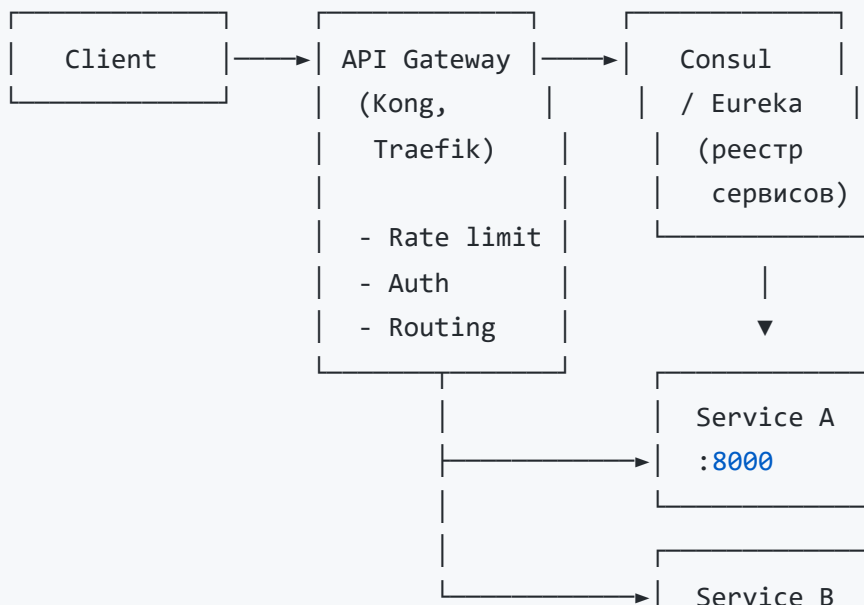
- Бинарный protobuf — компактнее JSON в 3–5 раз.
- HTTP/2 — мультиплексирование, streaming.
- Строгая типизация — контракт на уровне protobuf.

### 11.4.3. Service Discovery и API Gateway

**Service Discovery:** когда сервисов много, их адреса меняются (масштабирование, деплой).

Нужен реестр:

In [28]:





:8001

**API Gateway** — единая точка входа:

- **Маршрутизация:** `/api/v1/predict` -> `prediction-service`, `/api/v1/models` -> `model-service`.
- **Аутентификация:** проверка JWT один раз, а не в каждом сервисе.
- **Rate limiting:** глобальные лимиты.
- **SSL termination:** HTTPS наружу, HTTP внутри.

**Traefik** (рекомендация для Kubernetes):

```
In [29]: # IngressRoute для Traefik
apiVersion: traefik.io/v1alpha1
kind: IngressRoute
metadata:
  name: ml-api
spec:
  entryPoints:
    - websecure
  routes:
    - match: Host(`api.example.com`) && PathPrefix(`/v1/predictions`)
      kind: Rule
      services:
        - name: prediction-service
          port: 8000
      middlewares:
        - name: rate-limit
        - name: auth-forward
```

#### 11.4.4. CAP-теорема: фундаментальное ограничение

**CAP-теорема** (Brewer, 2000): в распределённой системе с репликацией данных невозможно одновременно гарантировать все три свойства:

- **Consistency** (консистентность): все узлы видят одни и те же данные в один момент.
- **Availability** (доступность): каждый запрос получает ответ (успех или ошибка).
- **Partition tolerance** (устойчивость к разделению): система работает при разрыве сети между узлами.

**Теорема:** при наличии сетевого разделения (*P*) система может гарантировать только *C* или *A*, но не оба.

**Интуиция:** сеть разделилась на две части. Клиент пишет в часть A. Если система доступна (A), часть B не узнает об этом writes — inconsistency. Если система консистентна (C), часть B должна отклонить чтение до синхронизации — недоступность.

**PACELC** — расширение CAP: даже без разделения (P) есть компромисс между **L**atency и **C**onsistency.

**Практические следствия:**

| Система   | Выбор                                 | Пример                                           |
|-----------|---------------------------------------|--------------------------------------------------|
| <b>CP</b> | Консистентность + Partition tolerance | PostgreSQL (синхронная реплика), etcd, ZooKeeper |
| <b>AP</b> | Доступность + Partition tolerance     | Cassandra, DynamoDB, Redis (async replication)   |
| <b>CA</b> | Только без разделений                 | Одноузловые системы (не распределённые)          |

**Для ML-сервиса:**

- **Метаданные моделей** (кто, когда, какую модель загрузил) — CP. Нельзя потерять или разойтись.
- **Результаты предсказаний** — AP с eventual consistency. Можно записать асинхронно, главное — не потерять.
- **Кэш фичей в Redis** — AP. При разделении отдаём stale данные, но сервис работает.

#### 11.4.5. Консенсус: Raft и Paxos — интуиция

**Консенсус** — задача согласования значения между узлами в присутствии отказов. Например: какой узел является лидером? Какое значение записать?

**Paxos** (Lamport, 1989) — классический, но сложный в реализации.

**Raft** (Ongaro, Ousterhout, 2014) — упрощённая альтернатива, используется в etcd, Consul, TiKV.

**Интуиция Raft:**

1. **Выборы лидера:** узлы голосуют. Побеждает тот, кто получил большинство. Лидер — единственный, кто принимает writes.

2. **Репликация лога:** лидер отправляет записи follower'ам. Запись считается подтверждённой, когда большинство узлов записало её в лог.
3. **Безопасность:** если лидер упал — новый лидер выбирается среди узлов с самым актуальным логом.

**Формально:** Raft гарантирует safety (не будет двух лидеров одновременно) и liveness (если большинство узлов живы — система прогрессирует) при  $f$  отказах из  $2f + 1$  узлов.

**Для чего это нужно ML-инженеру:** понимать, почему etcd (хранилище конфигурации Kubernetes) требует нечётного числа узлов (3, 5), и почему split-brain — смертельно.

## 11.5. REST API vs Event-Driven архитектура

### 11.5.1. Пространственная связанность (Spatial Coupling)

**REST требует знания адреса сервиса.** Клиент знает: «мне нужно отправить POST на `http://prediction-service:8000/predict` ». Если сервис переехал — клиент ломается. Если сервисов много — клиент должен знать все адреса.

**Event-Driven:** клиент публикует событие в брокер («нужно предсказание»). Кто обработает — неважно. Сервисы подписываются на интересующие их события.

```
In [30]:  REST:      Event-Driven:
          A → B      A → [Broker] ← B
          A → C      A → [Broker] ← C
          A → D      [Broker] ← D
```

**Вывод:** Event-Driven снижает пространственную связанность. Producer не знает о consumers.

### 11.5.2. Временная связанность (Temporal Coupling)

**REST требует одновременной доступности.** Если B недоступен во время вызова — запрос падает. Нет буферизации.

**Event-Driven:** событие публикуется в брокер и хранится там. Consumer может быть offline и обработать позже.

```
In [31]:  Время:      t1          t2          t3
          REST:      A → B (B упал) -> ошибка
          Event:      A → [Broker] -> (B offline) -> B просыпается -> обрабатывает
```

**Вывод:** Event-Driven снижает временную связанность. Producer и consumer не должны быть online одновременно.

### 11.5.3. Лавинные нагрузки (Thundering Herd)

**Проблема REST-polling:** 1000 клиентов опрашивают статус задачи каждую секунду. Это 1000 RPS на сервер, независимо от того, изменился ли статус.

**Event-Driven с backpressure:** клиенты подписаны на события. Событие рассылается только при изменении. Нагрузка пропорциональна числу изменений, а не числу клиентов.

In [32]:

Polling: 1000 клиентов × 1 RPS = 1000 RPS на сервер  
Events: 1 изменение × 1000 push'ей = 1000 сообщений, но только при изменении

**Backpressure** — способность consumer'a сигнализировать producer'у: «я не справляюсь, замедлись». В Kafka — это `max.poll.records` , `fetch.min.bytes` . В RabbitMQ — `prefetch count`.

### 11.5.4. Message Brokers: сравнение

| Брокер        | Модель                 | Персистентность | Масштабирование | Когда использовать                                 |
|---------------|------------------------|-----------------|-----------------|----------------------------------------------------|
| RabbitMQ      | AMQP, очереди, routing | Да (диск)       | Кластер         | Сложная routing, RPC, DLQ                          |
| Apache Kafka  | Log-based, partitions  | Да (репликация) | Горизонтальное  | High throughput, event sourcing, stream processing |
| NATS          | Pub/sub, req/reply     | Опционально     | Лёгкий          | Простота, low-latency, service mesh                |
| Redis Pub/Sub | Pub/sub                | Нет (in-memory) | Ограничено      | Простые уведомления, кэш-инвалидация               |

**Kafka — deep dive:**

Kafka моделирует поток событий как **append-only log** (журнал):

```
In [33]: Partition 0: [Event_0] -> [Event_1] -> [Event_2] -> [Event_3] -> ...
           ↑offset=0   ↑offset=1   ↑offset=2   ↑offset=3
```

- **Producer** пишет в конец log'a.
- **Consumer** читает с определённого offset'a.
- **Consumer Group**: несколько consumer'ов делят partition'ы между собой. Каждое событие обрабатывается ровно одним consumer'ом в группе.
- **Retention**: данные хранятся N дней независимо от того, прочитал ли их consumer.

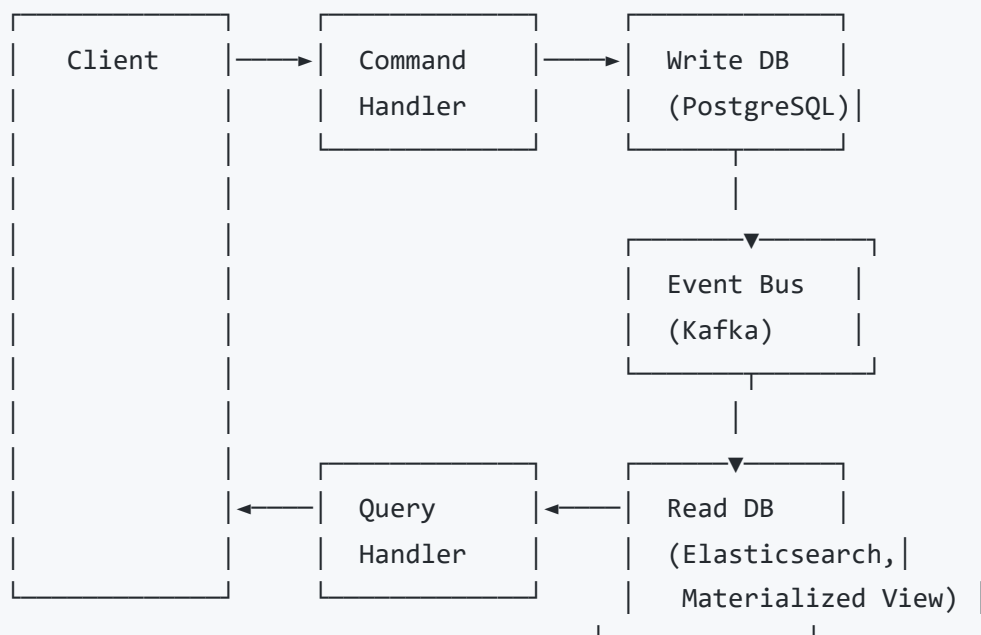
### Почему Kafka для ML:

- **Event sourcing**: история всех предсказаний — immutable log. Можно переиграть для анализа.
- **Stream processing**: Flink/Spark Streaming читают из Kafka, обрабатывают, пишут обратно.
- **Масштабирование**: миллионы событий в секунду на кластер.

### 11.5.5. CQRS: разделение команд и запросов

**CQRS** (Command Query Responsibility Segregation) — паттерн, где операции записи (commands) и чтения (queries) используют разные модели данных.

```
In [34]:
```



### Пример для ML-платформы:

- **Command**: `CreatePrediction` -> пишет в PostgreSQL, публикует `PredictionCreated`.

- **Query:** `GetPredictionsByUser` -> читает из Elasticsearch, где данные денормализованы и проиндексированы.
- **Sync:** consumer слушает `PredictionCreated` и обновляет Elasticsearch.

**Преимущества:**

- Write DB оптимизирована для консистентности (нормализация, транзакции).
- Read DB оптимизирована для скорости чтения (денормализация, индексы).
- Можно масштабать read и write независимо.

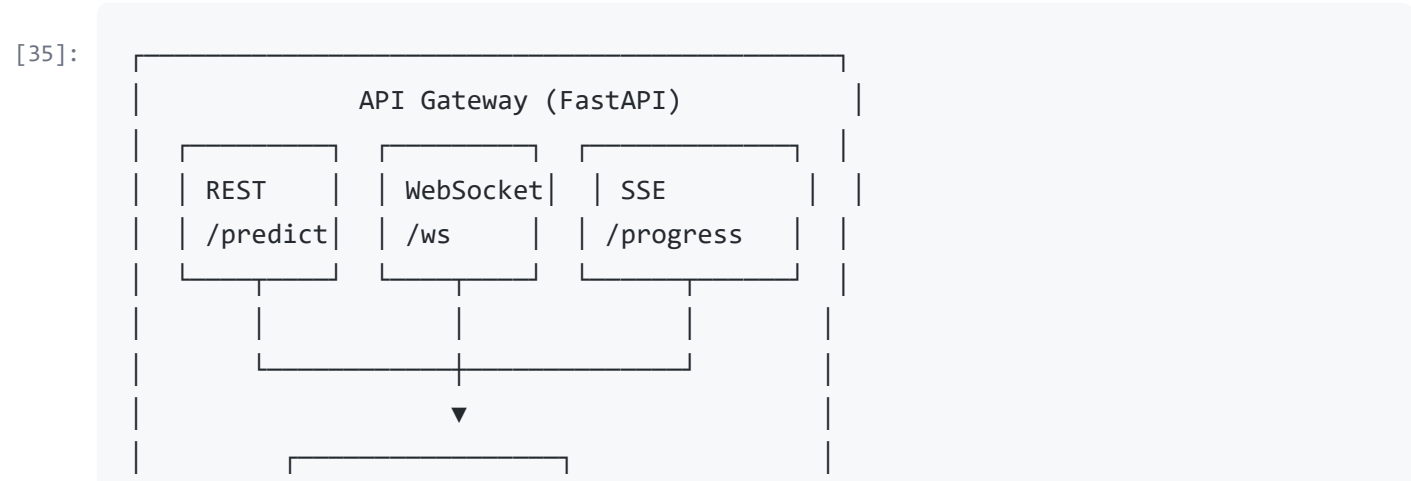
**Недостатки:**

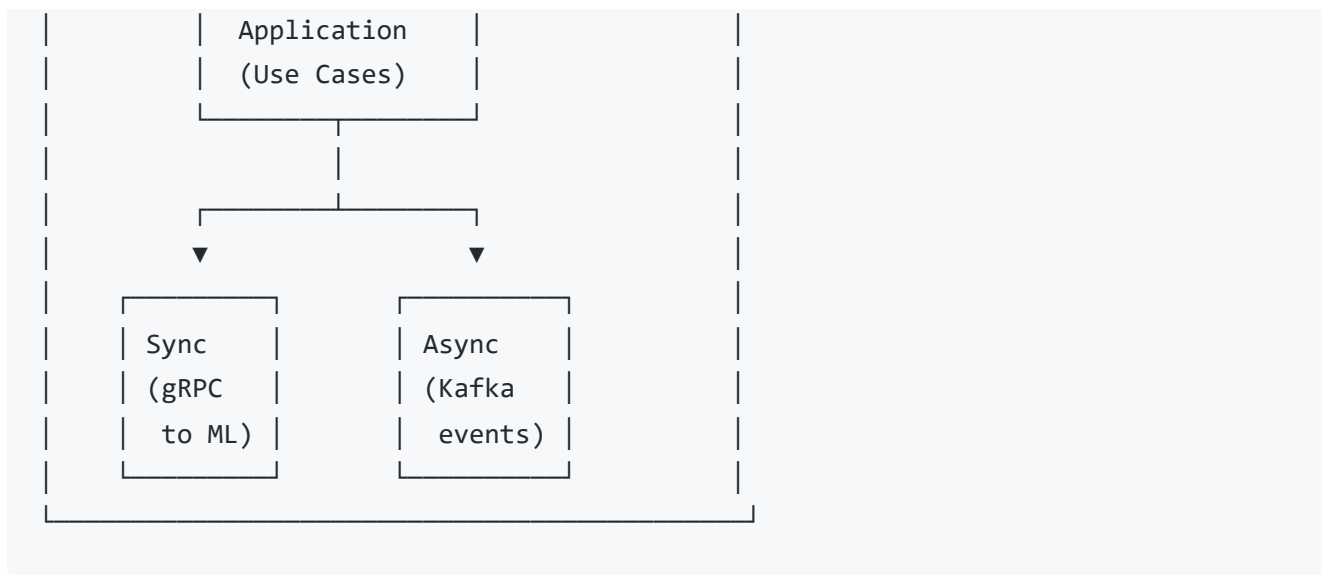
- Eventual consistency между write и read.
- Сложность синхронизации (outbox pattern помогает).

11.5.6. Выбор архитектуры под задачу

| Сценарий                                 | Архитектура           | Почему                                    |
|------------------------------------------|-----------------------|-------------------------------------------|
| <b>Real-time ML inference</b> (< 100 мс) | REST/gRPC, синхронный | Минимальная latency, нет overhead брокера |
| <b>Batch processing</b>                  | Event-Driven (Kafka)  | Накопление, buffering, replay             |
| <b>Long-running training jobs</b>        | Event-Driven + Saga   | Асинхронность, компенсации, monitoring    |
| <b>Real-time notifications</b>           | WebSocket / SSE       | Push, low latency                         |
| <b>Data pipeline</b>                     | Kafka + Flink         | Stream processing, event sourcing         |

**Гибридный подход (рекомендация):**





- **Синхронный путь:** запрос inference -> gRPC к ML Server -> ответ. Быстро, предсказуемо.
- **Асинхронный путь:** события (обучение завершено, модель обновлена) -> Kafka -> consumers (notification, analytics, cache invalidation).

## Итог модуля 11

| Концепция                       | Суть                                           | Практическое применение                                   |
|---------------------------------|------------------------------------------------|-----------------------------------------------------------|
| <b>Чистая архитектура</b>       | Слои с направленными внутри зависимостями      | Тестируемость, заменяемость БД/ML-сервисов                |
| <b>Protocol / ABC</b>           | Интерфейсы для инверсии зависимостей           | Domain Layer не знает об Infrastructure                   |
| <b>Circuit Breaker</b>          | Автоматическое отключение при отказах          | Защита от каскадных падений ML-сервисов                   |
| <b>Retry + Backoff + Jitter</b> | Повторные попытки с экспоненциальной задержкой | Устойчивость к временным сбоям сети                       |
| <b>Idempotency Key</b>          | Безопасные повторные запросы                   | Предотвращение двойного списания/сохранения               |
| <b>Outbox Pattern</b>           | Транзакционная публикация событий              | Гарантия доставки событий в Kafka                         |
| <b>Saga</b>                     | Распределённые транзакции через компенсации    | Многошаговые операции (оплата -> обучение -> уведомление) |

| Концепция                  | Суть                                | Практическое применение                           |
|----------------------------|-------------------------------------|---------------------------------------------------|
| <b>REST</b>                | Ресурсы, HTTP-методы, статус-коды   | Публичное API, CRUD-операции                      |
| <b>gRPC</b>                | Бинарный protobuf на HTTP/2         | Внутреннее взаимодействие, high throughput        |
| <b>Версионирование API</b> | URL / header / query param          | Совместимость со старыми клиентами                |
| <b>CAP-теорема</b>         | C, A, P — выбери два                | Понимание компромиссов БД и кэшей                 |
| <b>Raft/Paxos</b>          | Консенсус в распределённых системах | Почему etcd требует 3/5 узлов                     |
| <b>Event-Driven</b>        | Публикация/подписка через брокер    | Снижение пространственной и временной связанности |
| <b>CQRS</b>                | Разделение команд и запросов        | Оптимизация read/write под разные нагрузки        |
| <b>Kafka</b>               | Log-based messaging, partitions     | Event sourcing, stream processing, replay         |

В **Модуле 12** мы соберём всё вместе: построим полноценный ML-сервис от нуля до деплоя, применяя паттерны этого модуля на практике.